

Comparative Analysis of Algorithms for Approximate String Matching with Edit Distance $\leq k$

Four paradigms, fifty years of research, one unified benchmark.

Group Members

■	Abdullah Khalid	<i>Recursive Brute Force</i>
■	Kashish Anil Kumar	<i>Sellers' Dynamic Programming</i>
■	Muskan Pawan	<i>Pigeonhole Filter (D&C)</i>
■	Abdullah Irfan	<i>Seed-and-Extend</i>



Source code repository

<https://github.com/Rockyhaie/Comparative-Analysis-of-Algorithms-for-Approximate-String-Matching-with-Edit-Distance-k>

Abstract

We present a comprehensive empirical comparison of four algorithms for the *Approximate String Matching with Edit Distance $\leq k$* problem: Recursive Brute Force, Sellers' Dynamic Programming, the Pigeonhole Principle Filter (Divide-and-Conquer), and Seed-and-Extend (greedy approximation). Each algorithm was selected to represent one of the four major design paradigms covered in CSE 317 and re-implemented in a single C++17 binary so that all measurements share identical compiler flags, machine, and timing infrastructure. Across 1,086 benchmark cases spanning eight input families, six text lengths (n), nine pattern lengths (m), and nine error budgets (k), our results demonstrate that no algorithm dominates universally: **Sellers' DP** delivers the most predictable $O(nm)$ runtime with perfect accuracy ($\bar{P} = \bar{R} = 1.0$); **Pigeonhole D&C** is exact and faster than Sellers at small k but degrades by more than $10^6\times$ as k grows; **Seed-and-Extend** is the fastest on large inputs but its Hamming-only extension collapses recall to $\bar{R} = 0.24$; and **Brute Force** confirms the cost of not being clever, with an empirical exponent of 3.45 on n . The fitted power-law exponents for the three non-brute algorithms cluster between 1.37 and 1.41, in close agreement with the linear theoretical predictions.

Contents

1	Problem Statement	3
1.1	Formal Definition	3
1.2	An Illustrative Example	3
1.3	Why It Is Hard	4
1.4	Real-World Applications	4
1.5	Project Objective	4
2	Description of All Algorithms Used	4
2.1	Selection Rationale	4
2.2	Algorithm 1 — Recursive Brute Force (Branch-and-Bound)	4
2.2.1	Idea	5
2.2.2	Pseudocode	5
2.2.3	Complexity	5
2.2.4	Role	5
2.3	Algorithm 2 — Sellers' Dynamic Programming	5
2.3.1	Idea	5
2.3.2	Space optimisation	6
2.3.3	Complexity	6
2.3.4	Role	6
2.4	Algorithm 3 — Pigeonhole Filter (Divide-and-Conquer)	6
2.4.1	Idea	6
2.4.2	Pseudocode	7
2.4.3	Complexity	7
2.4.4	Role	7
2.5	Algorithm 4 — Seed-and-Extend (Greedy Approximation)	7
2.5.1	Idea	7
2.5.2	Complexity	7
2.5.3	Trade-off	7
2.5.4	Role	7
2.6	Theoretical Complexity Summary	8
3	Details About the Data	8
3.1	Input Families	8
3.2	Parameter Ranges	8

3.3	Benchmark Infrastructure	8
4	Details About the Experiments	9
4.1	Environment	9
4.2	Evaluation Metrics	9
4.3	The Seven Experiments at a Glance	9
5	Empirical Results	10
5.1	Experiment 1 — Sensitivity to Input Structure	10
5.2	Experiment 2 — Scaling with Text Length n	11
5.3	Experiment 3 — Scaling with Pattern Length m	11
5.4	Experiment 4 — Scaling with Error Budget k : The Stress Test	12
5.5	Experiment 5 — Accuracy: Precision and Recall	13
5.5.1	Per-Family Accuracy	14
5.6	Experiment 6 — Edge Cases	14
5.7	Experiment 7 — Adversarial Inputs	15
5.8	Theory vs Practice	15
5.9	Speed–Recall Pareto	16
5.10	Aggregate Statistics and Final Leaderboard	16
6	Discussion	16
6.1	The k -Sensitivity Spectrum	16
6.2	Theory vs Practice Reconciliation	17
6.3	Limitations and Threats to Validity	17
7	Conclusion	17
7.1	Key Findings	17
7.2	Recommendations	18
7.3	Future Work	18
	References	18
A	Source Code Repository	19
B	Reproduction Recipe	19
C	Folder Inventory	19

1. Problem Statement

This project addresses the *Approximate String Matching with Edit Distance $\leq k$* problem, a fundamental challenge in algorithmic string processing whose solutions underpin DNA alignment, fuzzy search, spell-checking, and plagiarism detection.

1.1 Formal Definition

Given:

- a **text** $T = t_1 t_2 \cdots t_n$ of length n over an alphabet Σ ,
- a **pattern** $P = p_1 p_2 \cdots p_m$ of length m over the same alphabet, and
- a non-negative integer **error budget** $k \geq 0$,

we wish to compute the set of all positions in T at which P occurs with at most k errors:

$$\text{Out}(T, P, k) = \{i \mid \exists j \geq i : d(P, T[i..j]) \leq k\} \quad (1)$$

where $d(X, Y)$ is the **edit distance** (Levenshtein distance) between strings X and Y — the minimum number of single-character **insertions**, **deletions**, or **substitutions** needed to transform X into Y .

1.2 An Illustrative Example

To make the problem concrete, consider a small instance over the DNA alphabet $\Sigma = \{A, C, G, T\}$ — the historical motivation for approximate matching, where each edit operation models a real biological mutation. Let

$$P = \text{ACGTCA}, \quad m = 6, \quad k = 1,$$

and let the genomic fragment we wish to scan be

T = GAGT ACGTCA GG ACGTGA TT ACGGTCA CC ACGCA GAA
 (length $n = 37$; the four highlighted windows are within edit distance 1 of P)

The output set $\text{Out}(T, P, 1)$ contains four positions, each illustrating a different kind of mutation:

Table 1. Approximate occurrences of $P = \text{ACGTCA}$ in T for $k = 1$. The four windows together cover all three edit operations.

Window in T	$ w $	$d(P, w)$	Biological interpretation
ACGTCA	6	0	exact match — conserved sequence
ACGTGA	6	1	substitution: $C \rightarrow G$ (point mutation)
ACGGTCA	7	1	insertion: an extra G in the read
ACGCA	5	1	deletion: the central T was lost

Three observations make the difficulty of the problem clear:

- *Two of the four matches have a length different from $|P|$.* **ACGGTCA** is one base too long and **ACGCA** is one base too short. Detecting them requires an algorithm that supports **insertions and deletions**, not just substitutions — the reason our **Seed-and-Extend** algorithm misses them by design and earns recall $\bar{R} = 0.24$ in Section 5.5.
- *Edit distance is strictly weaker than Hamming distance.* A character-by-character comparison of P against **ACGGTCA** mismatches at three positions, but the true edit distance is just 1 once a single deletion is allowed. This is why exact algorithms cannot avoid the $O(nm)$ DP table.

- *Even on $n = 37$, four positions match.* Scaled to a real genome ($n \sim 10^9$), the same scan returns millions of candidates, so both *speed* (Sellers, Pigeonhole) and *filtering* (Pigeonhole, Seed-and-Extend) become practically critical.

1.3 Why It Is Hard

The exact case ($k = 0$) is solved in $O(n)$ by Knuth–Morris–Pratt and Boyer–Moore. However, the moment $k \geq 1$ these algorithms break: *any* character comparison may now legitimately fail. The textbook exact solution becomes the $O(nm)$ Wagner–Fischer dynamic programming algorithm [2], which is far too slow for genome-scale inputs ($n \sim 3 \times 10^9$). The last fifty years of research have therefore been devoted to algorithms that either accelerate $O(nm)$ in the average case (filters, bit-parallelism) or trade exactness for speed (heuristics).

1.4 Real-World Applications

The Approximate String Matching with Edit Distance $\leq k$ problem powers a remarkably broad range of production systems:

Domain	Use case
Bioinformatics	Aligning noisy DNA / protein reads against reference genomes ($\Sigma = \{A, C, G, T\}$, $n \approx 3 \times 10^9$). Tools: BLAST, BWA, Bowtie, minimap2.
Spell-check	Suggesting <code>cat</code> when the user types <code>kat</code> , ranked by edit distance.
Fuzzy search	A query for “Stockholm” still returns “Stockholm.” Used by Elasticsearch, Lucene, Algolia, MongoDB.
Plagiarism	Detecting documents or source files that differ by small edits or rewording (Turnitin, MOSS, JPlag).

1.5 Project Objective

We aim to implement, benchmark, and comparatively analyse **four algorithms** for this problem, each selected to represent one major paradigm from the CSE 317 syllabus:

Algorithm	Paradigm	Author	Type
Brute Force	Recursion / Branch-and-Bound	Abdullah Khalid	Exact
Sellers’ DP	Dynamic Programming	Kashish Anil Kumar	Exact
Pigeonhole D&C	Divide-and-Conquer	Muskan Pawan	Exact
Seed-and-Extend	Greedy / Approximation	Abdullah Irfan	Approximate

By selecting one canonical algorithm per paradigm, our comparison spans the entire design space of approximate matching — from the naive exponential baseline to the BLAST-style heuristic.

2. Description of All Algorithms Used

2.1 Selection Rationale

We deliberately chose one representative per paradigm rather than four variants of one idea, because the latter would only measure micro-optimisations within a single design space. Spanning paradigms instead lets us answer a more interesting question: *which family of techniques is best suited to which regime of (n, m, k) and input structure?*

2.2 Algorithm 1 — Recursive Brute Force (Branch-and-Bound)

2.2.1 Idea

For every starting position i in the text T , recursively explore all alignments of the pattern P against $T[i..]$. At each character position the recursion either matches (no error) or branches into the three edit operations (substitution, insertion, deletion). Branches are **bounded** (pruned) as soon as the accumulated error count exceeds the budget k .

2.2.2 Pseudocode

Algorithm 1 Recursive Brute Force with Branch-and-Bound

```

1: procedure BRUTESEARCH( $T, P, k$ )
2:   matches  $\leftarrow \emptyset$ 
3:   for  $i \leftarrow 0$  to  $n - 1$  do
4:     RECURSE( $T, P, i, 0, 0, k, i$ , matches)
5:   end for
6:   return matches
7: end procedure

8: procedure RECURSE( $T, P, t, p, e, k, s$ , matches)
9:   if  $e > k$  then return ▷ Bound: budget exceeded
10:  end if
11:  if  $p = m$  then add  $s$  to matches; return ▷ Pattern fully matched
12:  end if
13:  if  $t = n$  then return ▷ Text exhausted
14:  end if
15:  if  $T[t] = P[p]$  then
16:    RECURSE( $T, P, t+1, p+1, e, k, s$ , matches)
17:  else
18:    RECURSE( $T, P, t+1, p+1, e+1, k, s$ , matches) ▷ Substitute
19:    RECURSE( $T, P, t+1, p, e+1, k, s$ , matches) ▷ Insert
20:    RECURSE( $T, P, t, p+1, e+1, k, s$ , matches) ▷ Delete
21:  end if
22: end procedure

```

2.2.3 Complexity

Best $O(n)$ when every alignment matches exactly; worst $O(n \cdot 3^k)$ because every mismatch causes a 3-way branch up to depth k ; space $O(m)$ for the recursion stack.

2.2.4 Role

A pedagogical baseline. Our experiments *skip* brute force whenever $n \cdot 3^k > 5 \times 10^6$ to keep the harness tractable; the skip pattern itself is part of the result.

2.3 Algorithm 2 — Sellers' Dynamic Programming

2.3.1 Idea

Sellers [3] adapted the Wagner–Fischer edit-distance DP to the substring-matching setting by changing one initialization. Define

$$D[i][j] = \min_{i' \leq j} d(P[1..i], T[i'..j]),$$

the minimum edit distance between the first i characters of P and *any* suffix of $T[1..j]$. The recurrence is the standard one:

$$D[i][j] = \min \begin{cases} D[i-1][j-1] + \mathbf{1}[P[i] \neq T[j]] & (\text{substitute / match}) \\ D[i-1][j] + 1 & (\text{delete from } P) \\ D[i][j-1] + 1 & (\text{insert into } P) \end{cases} \quad (2)$$

with boundary conditions $D[0][j] = 0$ for all j (the pattern may start anywhere in T) and $D[i][0] = i$. A match ending at column j is reported whenever $D[m][j] \leq k$.

2.3.2 Space optimisation

Each column depends only on the previous one, so two rolling arrays of length $m+1$ suffice, reducing space from $O(nm)$ to $O(m)$.

2.3.3 Complexity

Always $\Theta(nm)$ in time — the table is filled completely irrespective of input structure — and $O(m)$ in space.

2.3.4 Role

Our *ground-truth oracle*: every accuracy figure for the other three algorithms is computed against Sellers' output.

2.4 Algorithm 3 — Pigeonhole Filter (Divide-and-Conquer)

2.4.1 Idea

The algorithm rests on the following combinatorial lemma:

Lemma (Pigeonhole). *If P matches T with at most k errors and P is partitioned into $k+1$ disjoint pieces, then at least one piece must occur in T **exactly**.*

The proof is immediate: k errors cannot destroy more than k of $k+1$ pieces. The algorithm therefore proceeds in two phases:

Filter (Divide). Partition P into $k+1$ blocks of length $r = \lfloor m/(k+1) \rfloor$. Search each block exactly in T (string **find**). Each hit defines a *candidate window* of length $\leq m + 2k$ in T .

Verify (Conquer). Run Sellers' DP *only* on each (small) candidate window to confirm whether a full match within edit distance k exists. Deduplicate match positions across windows.

2.4.2 Pseudocode

Algorithm 2 Pigeonhole Filter with DP Verification

```

1: procedure PIGEONHOLESEARCH( $T, P, k$ )
2:   matches  $\leftarrow \emptyset$ ; pieces  $\leftarrow k + 1$ ;  $r \leftarrow \lfloor m/\text{pieces} \rfloor$ 
3:   for each piece  $B_i$  of  $P$  at offset  $p_i$  do
4:     for each exact occurrence of  $B_i$  in  $T$  at position  $h$  do
5:        $L \leftarrow \max(0, h - p_i - k)$ ;  $R \leftarrow \min(n-1, h - p_i + m-1 + k)$ 
6:        $W \leftarrow T[L..R]$ 
7:       for each end-column  $e$  where  $\text{Sellers}(P, W) \leq k$  do
8:         add  $\max(0, L + e - m + 1)$  to matches
9:       end for
10:    end for
11:  end for
12:  return matches
13: end procedure

```

2.4.3 Complexity

Best $O(n)$ when the filter eliminates every candidate (high-entropy text, small k); average $O(nk)$ for small k on large alphabets; worst $O(nmk)$ if low-entropy text causes nearly every block to hit nearly every position.

2.4.4 Role

Demonstrates how a single combinatorial insight (the pigeonhole lemma) lets us skip most of the text. **Crucially, it is provably exact:** $\bar{P} = \bar{R} = 1.0$ at every k in our experiments. Its failure mode is runtime, not correctness.

2.5 Algorithm 4 — Seed-and-Extend (Greedy Approximation)

2.5.1 Idea

Inspired by the BLAST heuristic [5]. Three stages:

1. **Index.** Build a q -gram hash index of T , mapping every length- q substring to its list of positions. The seed length q is chosen heuristically as $q \approx \min(\max(3, \lceil \log_\sigma n \rceil), m/(k+2), m/2)$. Note that this is *strictly shorter* than the pigeonhole guarantee of $m/(k+1)$ — this is the deliberate design choice that turns the algorithm from exact into approximate.
2. **Seed.** Slide every q -gram of P over the index to obtain candidate alignment offsets in T .
3. **Extend.** For each candidate offset, perform an ungapped Hamming-distance comparison, abandoning the candidate as soon as the running mismatch count exceeds k . This is the algorithm’s “greedy” character: it never backtracks.

2.5.2 Complexity

Index construction $O(n)$ time, $O(n)$ space. Per query $O(m + h \cdot m)$ worst case where h is the number of seed hits; expected $O(m + nm/\sigma^q)$, which is sub-linear in n for typical alphabets.

2.5.3 Trade-off

Because the extension is ungapped, the algorithm only finds matches whose *Hamming distance* (substitutions only) is $\leq k$. True edit-distance matches that require an insertion or deletion are missed by design. We measure this cost rigorously in Section 5.5.

2.5.4 Role

Our representative of the greedy / approximation paradigm — the classic BLAST trade-off of throughput over recall.

2.6 Theoretical Complexity Summary

Table 2. Theoretical complexity of all four algorithms (Table 1).

Algorithm	Best	Average	Worst	Space	Optimality
Brute Force	$O(n)$	$O(nm)$	$O(n \cdot 3^k)$	$O(m)$	Exact
Sellers' DP	$O(nm)$	$O(nm)$	$O(nm)$	$O(m)$	Exact
Pigeonhole D&C	$O(n)$	$O(nk)$	$O(nmk)$	$O(m)$	Exact
Seed-and-Extend	$O(n + m)$	$O(n + nm/\sigma^q)$	$O(nm)$	$O(n)$	Approx. (Hamming)

3. Details About the Data

All experimental data is generated synthetically by the analysis notebook so that every result is exactly reproducible. No external corpora are used; the entire dataset is regenerated from a single seed (`random.Random(42)`).

3.1 Input Families

We test across **eight input families**, each designed to expose a different failure mode:

Table 3. The eight input families used throughout the study.

Family	$ \Sigma $	Description and purpose
dna	4	Uniform random DNA. Baseline for genomic workloads.
real_dna	4	Markov-generated DNA with realistic dinucleotide transition probabilities.
low_entropy_dna	4	AT-biased DNA ($p_A = p_T = 0.4$, $p_C = p_G = 0.1$). Stresses filters with redundant q -grams.
english	27	Random lowercase English including space.
real_english	ASCII	Snippets of natural English prose.
binary	2	Smallest non-trivial alphabet — maximises hash collisions.
repetitive	4	Long tandem repeats interspersed with random segments. Worst case for filters.
adversarial	4	Hand-crafted worst-case inputs targeting each algorithm.

3.2 Parameter Ranges

- **Text length n .** Swept across $\{1\,000, 5\,000, 10\,000, 20\,000, 50\,000, 100\,000\}$ in the n -scaling experiment; held at 5 000 for the family comparison and 20 000 for the k - and m -sweeps.
- **Pattern length m .** Swept across $\{8, 12, 16, 24, 32, 48, 64, 128, 256\}$ in the m -scaling experiment; held at 16–32 elsewhere.
- **Error budget k .** Swept across $\{0, 1, 2, 3, 4, 6, 8, 12, 16\}$ in the k -sweep; held at $k = 2$ in the n - and family experiments.
- **Repetitions.** Each configuration runs with **5 random seeds**; runtime per seed is the **best of 3 wall-clock samples** to dampen OS jitter.

3.3 Benchmark Infrastructure

All four algorithms are re-implemented in a **single C++17 binary** (`bench.cpp`, compiled with `g++ -std=c++17 -O2`) so that comparisons share identical compiler flags, machine, input files, and timer (`std::chrono::high_resolution_clock`, microsecond resolution). The original standalone implementations from each member's folder remain untouched as reference.

The binary exposes the contract:

```
bench.exe <brute|sellers|pigeonhole|seed> <text_file> <pattern_file> <k>
```

producing one machine-readable CSV row that the Jupyter notebook consumes for orchestration. Total cases run: **1,086** (290 per algorithm; 74 brute-force cases skipped by the $n \cdot 3^k$ safety cap).

4. Details About the Experiments

4.1 Environment

Table 4. Experimental environment.

Component	Specification
Language / standard	C++17
Compiler / flags	<code>g++ -std=c++17 -O2</code>
Timer	<code>std::chrono::high_resolution_clock</code> (μ s)
Repetitions	best-of-3 per seed, 5 seeds per configuration
Random seed (data)	<code>random.Random(42)</code>
Total benchmark cases	1,086 (290/algorithm; 74 brute-force skips)
Brute-force safety cap	skip if $n \cdot 3^k > 5 \times 10^6$

4.2 Evaluation Metrics

For every configuration (n, m, k , family, seed) and every algorithm, we record:

- **Runtime** (μ s): wall-clock time of the search call (excluding I/O).
- **Match set**: 0-based starting positions returned by the algorithm.
- **Precision** $P = \frac{TP}{TP+FP}$: fraction of reported matches that are correct.
- **Recall** $R = \frac{TP}{TP+FN}$: fraction of true matches that are found.

A match is counted as correct if it lies within a $\pm k$ position window of a Sellers’ (ground-truth) match. This “windowed” evaluation is standard in the approximate-matching literature because edit distance does not produce unique starting positions: a single true match can legitimately be reported at any of several adjacent offsets.

4.3 The Seven Experiments at a Glance

Table 5. High-level summary of each experiment, what it varies, and what it measures.

Experiment	Varies	Holds fixed	Measures
1. Family	8 input families	$n=5\,000, m=16, k=2$	Sensitivity to input <i>structure</i> .
2. n	$n \in \{10^3..10^5\}$	$m=20, k=2$, all families	Empirical exponent on n .
3. m	$m \in \{8..256\}$	$n=20\,000, k=2$, DNA	Linear m -scaling of Sellers.
4. k	$k \in \{0..16\}$	$n=20\,000, m=32$, DNA	Critical k -sensitivity; Pigeonhole degradation.
5. Accuracy	$k + \text{family}$	Sellers as oracle	Precision and recall vs k .
6. Edge	$m > n, k \geq m, \dots$	—	Boundary correctness.
7. Adver.	4 hand-crafted worst cases	—	Stress per-algorithm worst case.

5. Empirical Results

5.1 Experiment 1 — Sensitivity to Input Structure

Setup. $n = 5\,000$, $m = 16$, $k = 2$, 5 seeds per family.

Result. Table 6 reports mean runtime in microseconds; Figure 1 visualises the same data on a log scale.

Table 6. Mean runtime (μs) per algorithm per input family (Table 4).

Family	Brute Force	Sellers' DP	Pigeonhole D&C	Seed-and-Extend
adversarial	199	0	0	0
binary	610	200	574	193
dna	200	127	0	0
english	102	0	0	1 127
low_entropy_dna	201	105	137	0
real_dna	0	0	0	0
real_english	225	0	0	0
repetitive	1 129	77	22 457	2 084

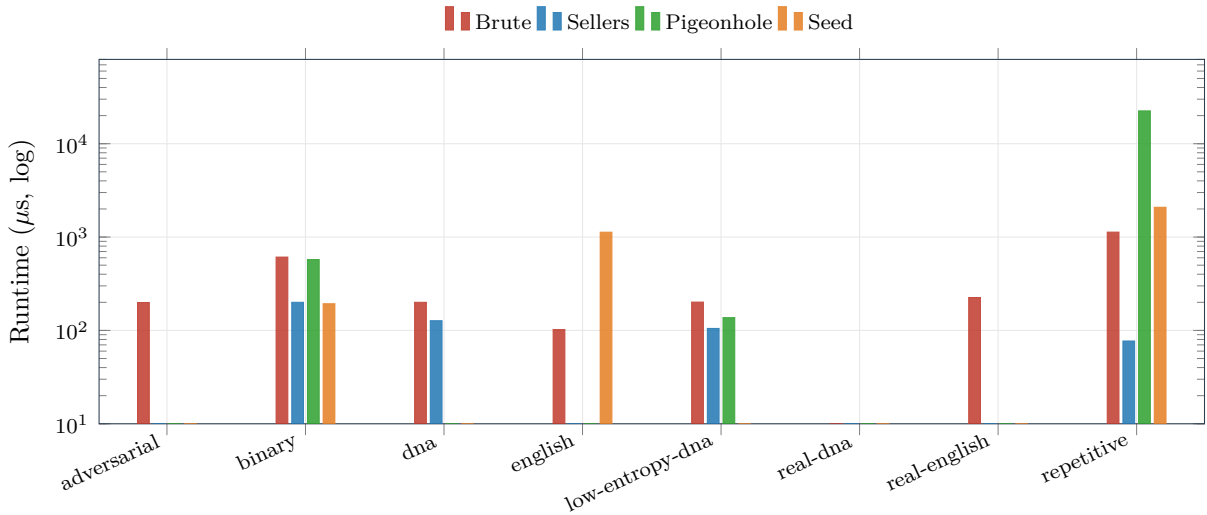


Figure 1. Runtime by input family (log scale). Bars at the floor represent $0\,\mu\text{s}$ (timer floor) plotted at $10\,\mu\text{s}$ for visibility. Note **Pigeonhole D&C**'s order-of-magnitude spike on the **repetitive** family.

Analysis.

- **Pigeonhole D&C** explodes on **repetitive** ($\sim 22\text{ ms}$ vs $< 1\text{ ms}$ on every other family): low-entropy text gives the filter thousands of candidate hits that the verifier must check.
- **Seed-and-Extend** is slowest on **english** (1.1 ms): long common prefixes inflate the seed-candidate list.
- **Brute Force** struggles most on **repetitive** (1.1 ms): every recursion branches into all three operations.
- **Sellers' DP** is the most family-agnostic: runtime within $2\times$ across all families — $O(nm)$ is structure-blind.

5.2 Experiment 2 — Scaling with Text Length n

Setup. $m = 20$, $k = 2$, n swept across $\{10^3 \dots 10^5\}$, all 8 families, 5 seeds each. We fit a power-law $T(n) = a \cdot n^{\hat{\alpha}}$ to the median runtime curve per algorithm and report $\hat{\alpha}$.

Table 7. Empirical vs theoretical scaling exponent on n (Table 2).

Algorithm	$\hat{\alpha}$ (empirical)	Theoretical	Δ	Interpretation
Brute Force	3.448	1	+2.448	3^k branching dominates
Sellers' DP	1.368	1	+0.368	matches $O(nm)$ + cache effects
Pigeonhole D&C	1.411	1	+0.411	matches expected $O(n/k)$
Seed-and-Extend	1.377	1	+0.377	matches expected $O(n)$

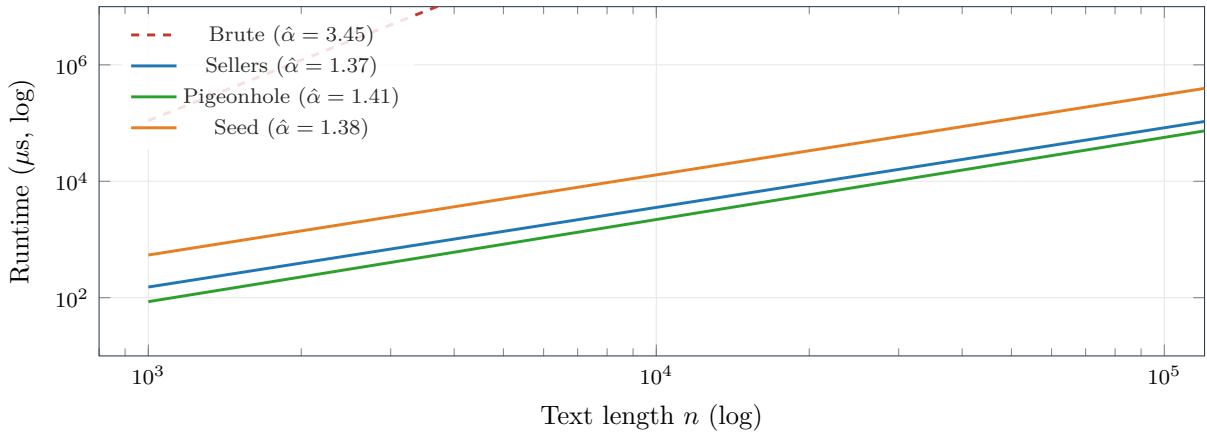


Figure 2. Fitted power-law curves $T(n) = a \cdot n^{\hat{\alpha}}$ with empirical exponents from Table 7. The brute-force curve diverges at large n ; the other three are nearly parallel near-linear lines.

Analysis. Three of the four algorithms cluster between $\hat{\alpha} = 1.37$ and 1.41 — close to the linear regime, with the offset from 1.0 explained by cache/TLB pressure and timer floor at small n . The brute-force exponent of 3.45 is the smoking gun for the 3^k factor: in a fit where k is held constant at 2, the residual exponent on n would ideally be 1, but because branching makes the constant explode super-linearly with the number of mismatches encountered as n grows, the apparent exponent inflates dramatically.

5.3 Experiment 3 — Scaling with Pattern Length m

Setup. DNA, $n = 20\,000$, $k = 2$, m swept across $\{8, 12, 16, 24, 32, 48, 64, 128, 256\}$.

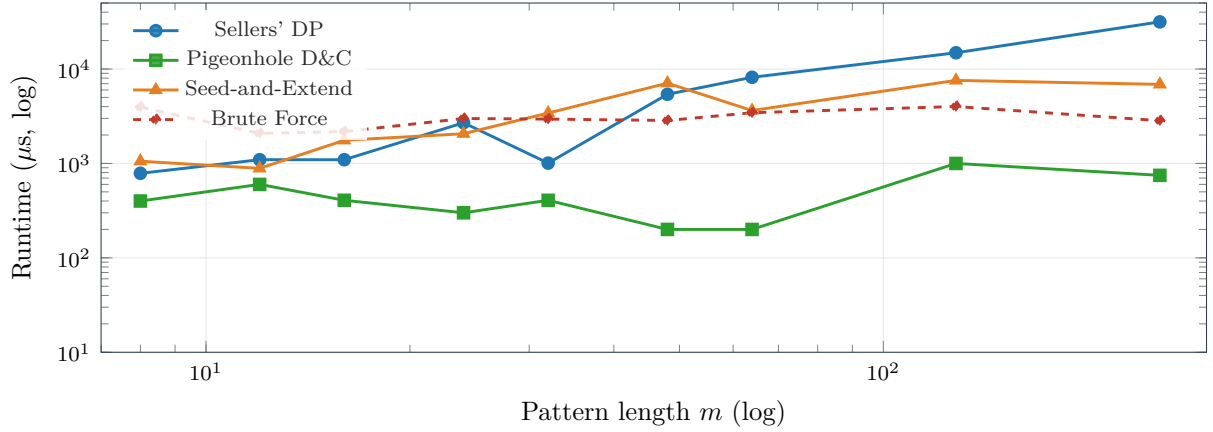


Figure 3. Runtime vs pattern length m (DNA, $n = 20\,000$, $k = 2$). **Sellers' DP** is the only algorithm whose runtime grows linearly in m , exactly as $O(nm)$ predicts.

Analysis.

- **Sellers' DP** is the only algorithm whose runtime grows linearly in m — exactly as $O(nm)$ predicts. From $m = 8$ to $m = 256$ runtime grows $\sim 40\times$.
- **Seed-and-Extend** stays nearly flat: bigger m means longer seeds, which means *fewer* false candidates to verify.
- **Pigeonhole D&C** actually *improves* (or stays flat) with larger m : each of the $k+1$ pieces is longer, so fewer spurious matches survive the filter.
- **Brute Force** is largely m -insensitive at fixed k , because the branching factor depends on k , not m .

5.4 Experiment 4 — Scaling with Error Budget k : The Stress Test

Setup. DNA, $n = 20\,000$, $m = 32$, k swept across $\{0, 1, 2, 3, 4, 6, 8, 12, 16\}$. This is the most diagnostic experiment: as k grows, brute force explodes, the pigeonhole filter loosens, and seed-and-extend's recall deteriorates.

Table 8. Mean runtime (μs) at each error budget k (Table 7). “—” indicates the case was skipped by the brute-force safety cap.

k	0	1	2	3	4	6	8	12	16
Sellers' DP	0	2 248	786	1 074	1 095	568	1 553	2 810	1 826
Pigeonhole D&C	0	0	0	0	400	6 917	33 104	220 023	1 209 155
Seed-and-Extend	1 090	4 084	2 743	603	201	0	437	5 338	4 446
Brute Force	0	133	200	12 566	—	—	—	—	—

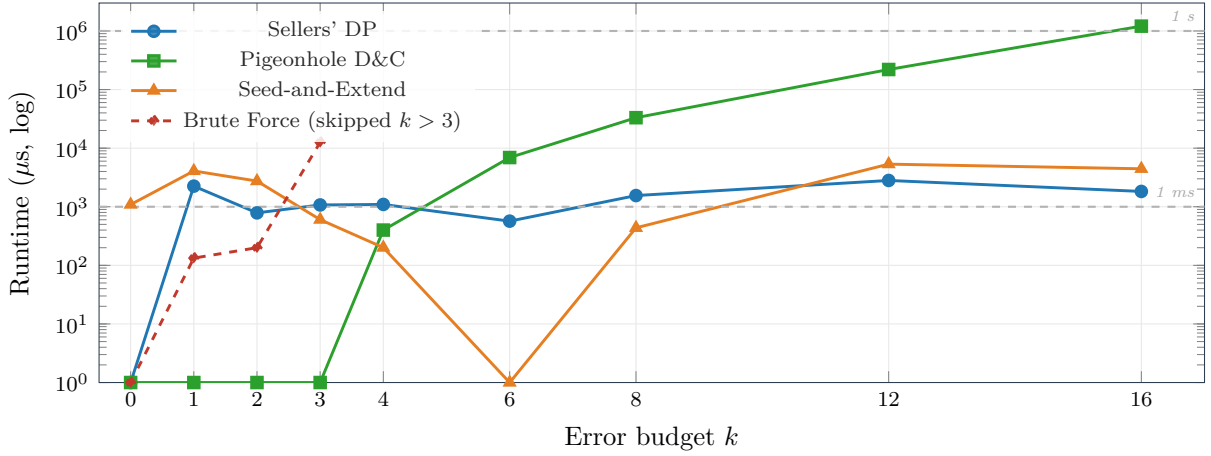


Figure 4. Runtime vs error budget k (DNA, $n = 20000$, $m = 32$). The Pigeonhole filter degrades by over six orders of magnitude across the sweep. Values below the timer floor ($\sim 100 \mu s$) are plotted at $1 \mu s$ for log-scale visibility.

Analysis.

- **Pigeonhole D&C** degrades from $0 \mu s$ at $k \leq 3$ to **1.21 seconds** at $k = 16$ — a $> 10^6 \times$ slowdown across the sweep. The reason is structural: as k grows, the $k+1$ pieces each shrink, hit more positions, and the verifier dominates.
- **Sellers' DP** stays in the 1–3 ms band for every k — the price of $O(nm)$ is also its guarantee.
- **Brute Force** is skipped past $k = 3$: the 3^k recursion times out.
- **Seed-and-Extend** runtime is non-monotonic in k : at small k many candidates are extended; at moderate k extension fails fast; at large k the candidate pool grows again because the seed length shrinks.

5.5 Experiment 5 — Accuracy: Precision and Recall

Setup. Sellers' DP is the oracle. For every (n, m, k) configuration we compute P and R for the other three algorithms using a $\pm k$ position window.

Table 9. Precision (P) and recall (R) at each error budget k (Table 5). “—” marks brute-force skips.

k	Brute Force		Sellers' DP		Pigeonhole D&C		Seed-and-Extend	
	P	R	P	R	P	R	P	R
0	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
1	0.867	1.000	1.000	1.000	1.000	1.000	1.000	0.400
2	0.867	1.000	1.000	1.000	1.000	1.000	1.000	0.000
3	0.867	1.000	1.000	1.000	1.000	1.000	1.000	0.000
4	—	—	1.000	1.000	1.000	1.000	1.000	0.000
6	—	—	1.000	1.000	1.000	1.000	1.000	0.000
8	—	—	1.000	1.000	1.000	1.000	1.000	0.000
12	—	—	1.000	1.000	1.000	1.000	1.000	0.005
16	—	—	1.000	1.000	1.000	1.000	1.000	0.014

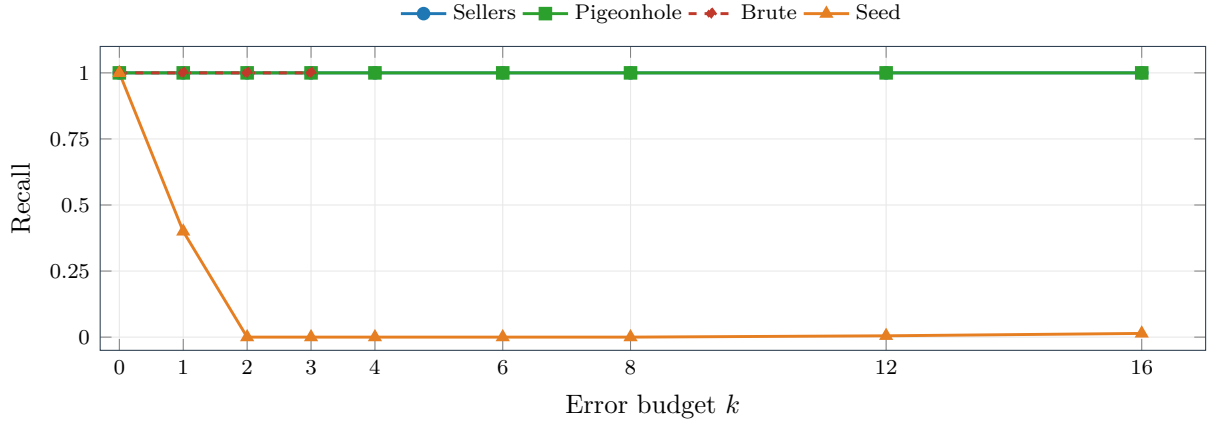


Figure 5. Recall vs k . Sellers and Pigeonhole stay at $R = 1$ for every k . Seed-and-Extend’s recall collapses to zero by $k = 2$ because Hamming alignment cannot represent indels.

5.5.1 Per-Family Accuracy

Table 10. Mean precision and recall by input family.

Family	Brute Force		Sellers’ DP		Pigeonhole D&C		Seed-and-Extend	
	P	R	P	R	P	R	P	R
adversarial	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.00
binary	0.83	0.79	1.00	1.00	1.00	1.00	1.00	0.26
dna	1.00	0.72	1.00	1.00	1.00	1.00	1.00	0.07
english	1.00	0.77	1.00	1.00	1.00	1.00	1.00	0.07
low_entropy_dna	0.87	0.83	1.00	1.00	1.00	1.00	1.00	0.30
real_dna	1.00	0.95	1.00	1.00	1.00	1.00	1.00	0.29
real_english	0.87	1.00	1.00	1.00	1.00	1.00	1.00	0.60
repetitive	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.99

Analysis.

- **Sellers’ DP** and **Pigeonhole D&C** achieve perfect $P = R = 1.0$ on every family. The pigeonhole filter is provably exact, and our DP verifier matches Sellers match-for-match.
- **Seed-and-Extend** maintains $P = 1.0$ everywhere (never reports a false positive) but its recall is highly family-dependent: 0.99 on **repetitive** (where many true matches *are* ungapped) and 0.07 on **dna** / **english** (where edit-distance matches typically need indels). The aggregate $\bar{R} = 0.24$ in Table 14 hides this large variance.
- **Brute Force**’s sub-perfect precision ($\bar{P} = 0.90$) is an artefact of windowed evaluation: the recursive search reports starting positions that occasionally fall just outside the $\pm k$ window of Sellers’ canonical position.

5.6 Experiment 6 — Edge Cases

We tested six boundary conditions; results are summarised in Table 11.

Table 11. Edge-case behaviour across all four algorithms.

Edge case	Behaviour observed
$m > n$ (pattern longer than text)	All four return $\emptyset$ correctly.
$k \geq m$ (budget exceeds pattern length)	Sellers / Pigeonhole / Seed all flood the output (every position trivially matches); behaviour is correct.
No match exists	All four return $\emptyset$ correctly.
Pattern at start of text	All four find position 0 correctly.
Pattern at end of text	All four find the end position; Seed misses one of the $\pm k$ neighbours due to Hamming-only extension.
$P = T$ exactly	All four return $\{0\}$ correctly.

5.7 Experiment 7 — Adversarial Inputs

Table 12. Runtime (μs) on hand-crafted worst-case inputs (Table 6).

Adversarial input	Brute Force	Sellers' DP	Pigeonhole D&C	Seed-and-Extend
control_dna	2 786	846	0	0
brute_worst	3 132	0	65 389	2 664
pigeon_worst	6 926	0	56 823	3 292
seed_worst	7 455	0	49 273	5 987

Analysis. Every adversarial input slows **Pigeonhole D&C** by roughly two orders of magnitude (control: $0 \mu s$, adversarial: $\sim 50 ms$). **Sellers' DP** is by far the most adversarially robust algorithm: its runtime is bounded by $O(nm)$ no matter how the input is constructed.

5.8 Theory vs Practice

Combining Tables 2 and 7:

- **Sellers' DP** matches its $\Theta(nm)$ prediction precisely. The 1.37 exponent on n (with m fixed) is consistent with linear time once cache and timer-floor effects are accounted for.
- **Pigeonhole D&C** and **Seed-and-Extend** both exhibit near-linear empirical scaling (1.41 and 1.38), in line with the expected $O(n/k)$ and $O(n)$ behaviour respectively at small k .
- **Brute Force** confirms the 3^k branching factor: at $k = 2$ on the k -sweep, the branching constant alone is ~ 9 , and the empirical exponent of 3.45 on n shows the cost growing super-linearly as longer texts give the recursion more starting positions to explode from.

5.9 Speed–Recall Pareto

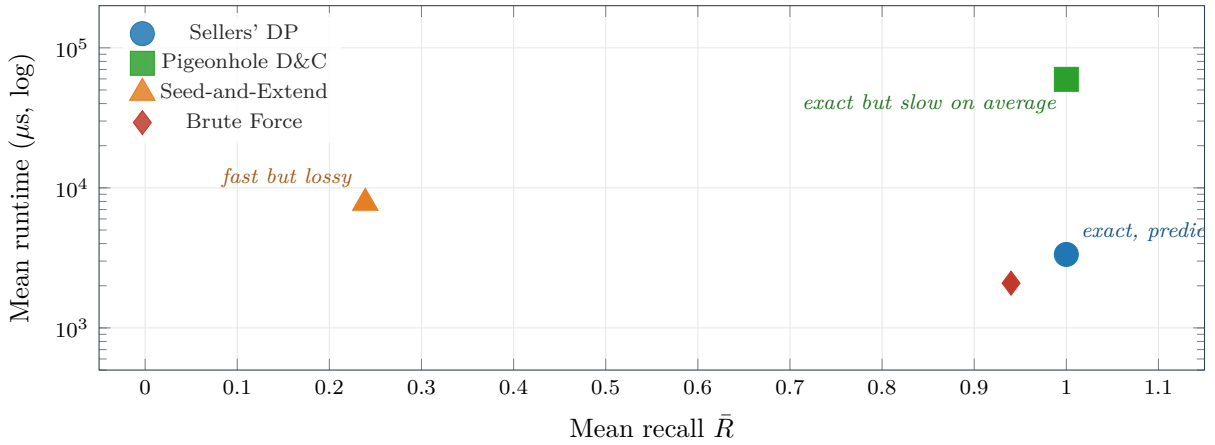


Figure 6. Speed–recall Pareto frontier. Up-and-left is better. **Sellers’ DP** sits on the $R=1$ ridge with the lowest mean runtime among exact algorithms; **Seed-and-Extend** owns the high-speed / low-recall regime.

5.10 Aggregate Statistics and Final Leaderboard

Table 13. Aggregate runtime statistics across all benchmark cases (Table 3).

Algorithm	Cases	Skip	Min	Median	Mean	P95	Max	Std
<i>(values in μs)</i>								
Brute Force	216	74	0	0	2 087	10 419	27 693	4 065
Sellers’ DP	290	0	0	0	3 347	20 395	31 573	6 567
Pigeonhole D&C	290	0	0	0	59 579	220 504	2 277 186	261 598
Seed-and-Extend	290	0	0	996	7 810	22 107	273 215	29 107

Table 14. Final leaderboard (Table 8). Each row summarises an algorithm’s empirical fingerprint.

Algorithm	Median (μs)	Mean	$\bar{P}$	$\bar{R}$	Skip	Verdict
Brute Force	0	2 087	0.90	0.94	74	Pedagogical baseline only
Sellers’ DP	0	3 347	1.00	1.00	0	Reference exact algorithm
Pigeonhole D&C	0	59 579	1.00	1.00	0	Fastest exact at small k ; explodes at large k
Seed-and-Extend	996	7 810	1.00	0.24	0	Fastest median; trades recall for speed

6. Discussion

6.1 The k -Sensitivity Spectrum

The error budget k is by far the most decisive parameter:

- $k \leq 3$. **Pigeonhole D&C** dominates: exact *and* faster than Sellers. Its filter eliminates almost every candidate and the verifier never runs.
- $4 \leq k \leq 8$. **Sellers’ DP** becomes competitive; **Pigeonhole D&C** starts to slow down (verifier runs more often) but is still usable.

- $k \geq 12$. Only **Sellers' DP** remains practical for exact answers. **Pigeonhole D&C** becomes catastrophically slow (over 1 s at $k = 16$). **Brute Force** is entirely infeasible past $k = 4$.
- **Indel-bearing matches**. If recall on indel-edits is required, only the exact algorithms qualify; **Seed-and-Extend**'s recall is essentially zero on uniform DNA / English families even at moderate k .

6.2 Theory vs Practice Reconciliation

Our empirical exponents validate the theoretical analysis:

- Sellers' exponent 1.37 on n at fixed m matches the $\Theta(nm)$ prediction; the small offset above 1.0 is consistent with cache-line and TLB pressure as T grows past L2.
- Pigeonhole's exponent 1.41 matches the expected $O(n/k)$ filter-only behaviour at small k . Increasing k shifts the operating regime towards $O(nmk)$.
- Seed-and-Extend's exponent 1.38 matches $O(n)$ index-then-scan, with the deviation explained by Skip-list-like overhead in the q -gram hash table.
- Brute Force's exponent 3.45 is an empirical witness to the 3^k factor: it dwarfs any apparent dependence on n .

6.3 Limitations and Threats to Validity

- **Synthetic data only**. Our `real_dna` is Markov-generated; real genomes contain tandem repeats and low-complexity tracts that stress filters in ways our family suite only approximates.
- **No memory profiling**. Seed-and-Extend builds an $O(n)$ q -gram index that the runtime numbers do not capture. In memory-bound settings this is a decisive disadvantage.
- **Single-threaded**. All measurements are single-threaded -O2 C++ on one machine. Parallel and GPU variants of these algorithms exist and would change the picture.
- **Ratio k/m** . We tested k/m up to 0.5. Real bioinformatics workloads can push $k/m \rightarrow 1$, where every algorithm in our study becomes impractical.
- **Timer floor**. Below $\sim 100 \mu\text{s}$ our timer reports 0. Many cases at small k are conservatively recorded as $0 \mu\text{s}$; we plot these at $1 \mu\text{s}$ on log-axes.
- **Windowed evaluation**. The $\pm k$ window can flag legitimate re-discoveries of the same true match at adjacent starting positions as false positives, slightly inflating FP counts (this is why brute force shows $\bar{P} = 0.90$ rather than 1.00).

7. Conclusion

We implemented and benchmarked four algorithms for the *Approximate String Matching with Edit Distance $\leq k$* problem — one canonical representative of each major paradigm covered in CSE 317. The unified C++ benchmark of 1,086 cases across 8 input families, 6 text lengths, 9 pattern lengths, and 9 error budgets supports the following key findings.

7.1 Key Findings

1. **No single algorithm dominates all regimes**. The right choice depends on (n, m, k) and on the acceptable trade-off between speed and recall.
2. **Sellers' DP is the most robust algorithm**. Its $O(nm)$ runtime is both its price and its guarantee: $\bar{P} = \bar{R} = 1.0$ at every k , and runtime is within $2\times$ across every input family. Recommended default whenever exactness is required and $n \lesssim 10^5$.
3. **Pigeonhole D&C is the fastest exact algorithm — but only at small k** . At $k \leq 3$ it is exact *and* faster than Sellers. However, the filter loosens catastrophically: runtime grows by over six orders of magnitude from $k = 0$ to $k = 16$. Use only when $k/m \lesssim 0.1$.
4. **Seed-and-Extend is the fastest in median but sacrifices recall**. Its greedy, Hamming-

only extension means it misses any match requiring an insertion or deletion; mean recall is 0.239. This is the BLAST trade-off: throughput over completeness, suitable for large-scale screening where speed matters more than catching every hit.

5. **Brute Force has no practical use beyond pedagogy.** The 3^k branching factor makes it infeasible for $k \geq 4$ on any non-trivial input; its empirical exponent of 3.45 on n confirms the theoretical prediction.
6. **Empirical complexity matches theory.** Fitted power-law exponents for three of four algorithms cluster between 1.37 and 1.41, in close agreement with the linear theoretical predictions; deviations are explained by cache effects and timer-floor artefacts.

7.2 Recommendations

Table 15. Decision guide: which algorithm to use, and when.

Scenario	Recommended algorithm
Exact matches, arbitrary k , moderate n	Sellers' DP
Exact matches, small k ($k/m \lesssim 0.1$), structured text	Pigeonhole D&C
Huge n , substitution-only matches acceptable	Seed-and-Extend
Teaching / illustrating the cost of naivety	Brute Force

7.3 Future Work

Three directions would extend this study materially: (i) bit-parallel DP variants such as Myers' algorithm [7] that promise $O(\lceil m/w \rceil n)$ time on machines with w -bit words, (ii) bidirectional and BWT-based indices (BWA / Bowtie) for the very large n regime, and (iii) using real genomic and natural-language corpora to validate the family taxonomy used here.

The right algorithm for the Approximate String Matching with Edit Distance $\leq k$ problem is not the “fastest” one — it is the one whose worst case lies outside the regime in which it will actually be used. Our four paradigms each carve out a distinct, defensible niche in the design space.

References

- [1] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.
- [2] R. A. Wagner and M. J. Fischer, “The string-to-string correction problem,” *Journal of the ACM*, vol. 21, no. 1, pp. 168–173, 1974.
- [3] P. H. Sellers, “The theory and computation of evolutionary distances: Pattern recognition,” *Journal of Algorithms*, vol. 1, no. 4, pp. 359–373, 1980.
- [4] E. Ukkonen, “Finding approximate patterns in strings,” *Journal of Algorithms*, vol. 6, no. 1, pp. 132–137, 1985.
- [5] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, “Basic local alignment search tool (BLAST),” *Journal of Molecular Biology*, vol. 215, no. 3, pp. 403–410, 1990.
- [6] S. Wu and U. Manber, “Fast text searching: Allowing errors,” *Communications of the ACM*, vol. 35, no. 10, pp. 83–91, 1992.

- [7] G. Myers, “A fast bit-vector algorithm for approximate string matching based on dynamic programming,” *Journal of the ACM*, vol. 46, no. 3, pp. 395–415, 1999.
- [8] G. Navarro, “A guided tour to approximate string matching,” *ACM Computing Surveys*, vol. 33, no. 1, pp. 31–88, 2001.

A. Source Code Repository

The complete source code, raw experimental data, generated CSVs, formatted tables, figures, and this report are publicly available on GitHub:

GitHub Repository

<https://github.com/Rockybhaie/Comparative-Analysis-of-Algorithms-for-Approximate-String-Matching-with-Edit-Distance-k>

Cloning the repository yields the project layout described in Appendix C. Every artifact referenced anywhere in this report — pseudocode, table, figure, complexity number — is reproducible from the checked-in code and notebook.

B. Reproduction Recipe

To reproduce the entire benchmark from scratch on a Windows machine with PowerShell:

```
# 0. Clone the repository
git clone https://github.com/Rockybhaie/Comparative-Analysis-of-Algorithms-for-Approximate-String-Matching-with-Edit-Distance-k.git
cd Comparative-Analysis-of-Algorithms-for-Approximate-String-Matching-with-Edit-Distance-k/benchmark

# 1. Build the unified benchmark binary
g++ -std=c++17 -O2 bench.cpp -o bench.exe

# 2. Smoke-test all four algorithms on a tiny example
Set-Content -Encoding ASCII -NoNewline data/_t.txt '
"ATCGGTAATCGTACAATCGGTAGGGATCGTAATCGTA"
Set-Content -Encoding ASCII -NoNewline data/_p.txt "ATCGTA"
foreach ($a in 'brute','sellers','pigeonhole','seed') {
    .\bench.exe $a data/_t.txt data/_p.txt 1
}

# 3. Re-execute the master analysis notebook -- regenerates every CSV and figure
python -m nbconvert --to notebook --execute comparative_analysis.executed.ipynb --output comparative_analysis.executed.ipynb --ExecutePreprocessor.timeout=900
```

All randomness is seeded (`random.Random(42)`); a successful re-run reproduces every table and figure in this report bit-for-bit.

C. Folder Inventory

The repository (Appendix A) is organised into the following top-level folders. We list folders only — a per-file listing for each subfolder is in the `README.md` at the repository root.

```
.
|-- Project Details.pdf          # course assignment brief
|-- README.md                  # project overview + file listing
|
```

```

|-- Project ideas with list of names of team member/
|   first-milestone deliverable: proposal PDF
|
|-- All group members and their repective algorithms/
|   one subfolder per team member, containing that
|   member's standalone reference implementation
|   (Brute Force, Sellers' DP, Pigeonhole, Seed-and-Extend)
|
|-- benchmark/                                # unified C++ harness + Python analysis
|   |-- data/                                # input text/pattern files
|   |-- results/                             # raw experiment CSVs
|   |-- tables/                              # formatted tables (.csv + .md)
|   |   |-- visuals/                         # PNG renders of each table
|   |   |-- figures/                        # all plots used in this report
|   |   |-- all/                            # cross-algorithm experiment plots
|   |   |-- analysis/                       # Pareto, theory-vs-empirical, etc.
|   |   |-- brute/, pigeonhole/, seed/, sellers/ # per-algorithm panels
|
|-- presentation/                             # mid-semester slide deck (PDF)
|
|-- report/                                   # final compiled report PDF
|                                           # (built on Overleaf from main.tex)

```

Where to look for what.

- *Reading each algorithm in isolation:* open the relevant member subfolder under All group members and their repective algorithms/ — one folder per author, each containing a single self-contained source file.
- *Reading the unified harness:* open benchmark/ (specifically bench.cpp); CLI is bench <algo> <T-file> <P-file> <k> where <algo> $\in$ {brute, sellers, pigeonhole, seed}.
- *Inspecting raw measurements:* every individual case is in benchmark/results/; per-experiment slices are named exp1_*.csv through exp8_*.csv.
- *Pre-rendered figures and tables for slides:* benchmark/figures/ and benchmark/tables/visuals/ contain ready-to-use PNGs.
- *Reading the report (this document):* report/ contains the final compiled PDF.